

M3_ML_classic

February 23, 2026

```
[3]: # =====
# Standalone: ChEMBL M3 ML pipeline (FP + physchem) WITHOUT OOP
# 1) Scaffold-CV benchmark: FP-only vs FP+physchem
# 2) Save final model (joblib) picklable
# 3) Proper scaffold-CV error analysis (re-fit per fold) no leakage
# - misclassifications per compound
# - per-scaffold performance
# - hard scaffolds report
#
# Requirements (in openms_env):
# - rdkit
# - numpy, pandas
# - scikit-learn
# - joblib
# =====
"""
Data
    Scaffold grouping
    GroupKFold splitting
        Train
        Test

    Feature generation (inside loop)
    Scaling
    Logistic Regression
    Metrics
"""

import warnings
warnings.filterwarnings("ignore")

import os
import numpy as np
import pandas as pd
import joblib

from sklearn.pipeline import Pipeline
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold
from sklearn.metrics import (
    roc_auc_score,
    average_precision_score,
    matthews_corrcoef,
    balanced_accuracy_score,
)

from rdkit import Chem
from rdkit.Chem.Scaffolds import MurckoScaffold
from rdkit.Chem import Descriptors
from rdkit.DataStructs import ConvertToNumpyArray
from rdkit.Chem import rdFingerprintGenerator

# -----
# USER SETTINGS
# -----
DATA_PATH = r"C:
↳\Users\Besitzer\Desktop\M3_Project\M3_databases\ChEMBL_M3_consensus_labels_more_negatives_w
↳csv"

OUT_DIR = r"C:\Users\Besitzer\Desktop\M3_Project\M3_databases"
SAVE_MODEL_PATH = os.path.join(OUT_DIR, "m3_fp_physchem_scaffoldcv.joblib")
MISCLASS_PATH = os.path.join(OUT_DIR, "scaffold_cv_misclassifications.csv")
SCAF_SUM_PATH = os.path.join(OUT_DIR, "scaffold_performance_summary.csv")
HARD_SCAF_PATH = os.path.join(OUT_DIR, "hard_scaffolds.csv")

POS_LABELS = {"active", "active_single"}
NEG_LABELS = {"inactive", "inactive_single"}

WEIGHTS = {
    "active": 1.0,
    "inactive": 1.0,
    "active_single": 0.5,
    "inactive_single": 0.7,
}

# fingerprint settings
FP_RADIUS = 2
FP_NBITS = 2048
FP_USE_CHIRALITY = True
FP_USE_FEATURES = False # feature invariants => FCFP-like

# scaffold CV

```

```

N_SPLITS = 10

# choose final model type
FINAL_USE_PHYSCHEM = True    # True => FP+physchem, False => FP-only

def make_clf():
    return LogisticRegression(
        max_iter=5000,
        solver="liblinear",
        class_weight="balanced",
    )

# =====
# 1) LOAD + FILTER DATA
# =====
df = pd.read_csv(DATA_PATH)

required = {"molecule_chembl_id", "smiles", "consensus_label"}
missing = required - set(df.columns)
if missing:
    raise ValueError(f"Missing required columns in input CSV: {missing}")

df = df[df["consensus_label"].isin(POS_LABELS | NEG_LABELS)].copy()
df = df.dropna(subset=["smiles"]).copy()
df["smiles"] = df["smiles"].astype(str).str.strip()
df = df[df["smiles"] != ""].copy()

df["y"] = df["consensus_label"].isin(POS_LABELS).astype(int)
df["w"] = df["consensus_label"].map(WEIGHTS).astype(float)

print("Loaded & filtered:", df.shape)
print(df["consensus_label"].value_counts())

# =====
# 2) SCAFFOLDS (GROUPS)
# =====
def smiles_to_scaffold(smi: str) -> str:
    mol = Chem.MolFromSmiles(smi)
    if mol is None:
        return ""
    scaff = MurckoScaffold.GetScaffoldForMol(mol)
    if scaff is None:
        return ""
    return Chem.MolToSmiles(scaff, isomericSmiles=False)

```

```

df["scaffold"] = df["smiles"].apply(smiles_to_scaffold)
bad = df["scaffold"].eq("")
if bad.any():
    print(f"Dropping {bad.sum()} rows with invalid SMILES/scaffold.")
    df = df[~bad].copy()

X_smiles = df["smiles"].values
y = df["y"].values.astype(int)
w = df["w"].values.astype(float)      # weights
groups = df["scaffold"].values

cv = GroupKFold(n_splits=N_SPLITS)

# =====
# 3) FEATURE FUNCTIONS (NO OOP)
# =====
def morgan_fp_matrix(smiles_list, radius, n_bits, use_chirality, use_features):
    """
    Return dense FP matrix (n_samples, n_bits) float32.
    NOTE: generator is created inside -> pickling safe.
    """
    smiles_list = np.asarray(smiles_list, dtype=object)

    atom_inv_gen = None
    if use_features:
        atom_inv_gen = rdFingerprintGenerator.GetMorganFeatureAtomInvGen()

    gen = rdFingerprintGenerator.GetMorganGenerator(
        radius=radius,
        fpSize=n_bits,
        includeChirality=use_chirality,
        atomInvariantsGenerator=atom_inv_gen,
    )

    X = np.zeros((len(smiles_list), n_bits), dtype=np.float32)
    tmp = np.zeros((n_bits,), dtype=np.int8)

    for i, smi in enumerate(smiles_list):
        mol = Chem.MolFromSmiles(str(smi))
        if mol is None:
            continue
        bv = gen.GetFingerprint(mol)
        tmp[:] = 0
        ConvertToNumpyArray(bv, tmp)

```

```

        X[i, :] = tmp

    return X

def physchem_matrix(smiles_list):
    """
    Compact physchem set (6 features):
    MolWt, MolLogP, HBD, HBA, TPSA, NumRotatableBonds
    """
    smiles_list = np.asarray(smiles_list, dtype=object)
    X = np.zeros((len(smiles_list), 6), dtype=np.float32)

    for i, smi in enumerate(smiles_list):
        mol = Chem.MolFromSmiles(str(smi))
        if mol is None:
            X[i, :] = np.nan
            continue
        X[i, 0] = Descriptors.MolWt(mol)
        X[i, 1] = Descriptors.MolLogP(mol)
        X[i, 2] = Descriptors.NumHDonors(mol)
        X[i, 3] = Descriptors.NumHAcceptors(mol)
        X[i, 4] = Descriptors.TPSA(mol)
        X[i, 5] = Descriptors.NumRotatableBonds(mol)

    if np.isnan(X).any():
        med = np.nanmedian(X, axis=0)
        r, c = np.where(np.isnan(X))
        X[r, c] = med[c]

    return X

def build_X(smiles_list, use_physchem: bool):
    X_fp = morgan_fp_matrix(
        smiles_list,
        radius=FP_RADIUS,
        n_bits=FP_NBITS,
        use_chirality=FP_USE_CHIRALITY,
        use_features=FP_USE_FEATURES,
    )
    if not use_physchem:
        return X_fp

    X_pc = physchem_matrix(smiles_list)
    return np.hstack([X_fp, X_pc]).astype(np.float32)

```

```

def make_pipe():
    # IMPORTANT:
    # - with dense FP, StandardScaler(with_mean=False) keeps sparse-friendly
    ↪ semantics,
    # but works fine for dense too and matches your previous choice.
    return Pipeline([
        ("scaler", StandardScaler(with_mean=False)),
        ("clf", make_clf()),
    ])

# =====
# 4) SCAFFOLD-CV EVALUATION (FP-only vs FP+physchem)
# =====
def eval_mode(name: str, use_physchem: bool):
    pipe = make_pipe()
    metrics = {"roc_auc": [], "pr_auc": [], "bal_acc": [], "mcc": []}
    print(f"\n=== {name} ===")

    for fold, (train_idx, test_idx) in enumerate(
        cv.split(X_smiles, y, groups=groups),
        start=1):

        X_train = build_X(X_smiles[train_idx], use_physchem=use_physchem)
        X_test = build_X(X_smiles[test_idx], use_physchem=use_physchem)

        y_train, y_test = y[train_idx], y[test_idx]
        w_train = w[train_idx]

        fold_pipe = make_pipe()
        fold_pipe.fit(X_train, y_train, clf__sample_weight=w_train)
        proba = fold_pipe.predict_proba(X_test)[: , 1]
        pred = (proba >= 0.5).astype(int)

        roc = roc_auc_score(y_test, proba)
        pr = average_precision_score(y_test, proba)
        bal = balanced_accuracy_score(y_test, pred)
        mcc = matthews_corrcoef(y_test, pred)

        metrics["roc_auc"].append(roc)
        metrics["pr_auc"].append(pr)
        metrics["bal_acc"].append(bal)
        metrics["mcc"].append(mcc)

    print(
        f"[Fold {fold}] ROC-AUC={roc:.3f} | PR-AUC={pr:.3f} | "

```

```

        f"BalAcc={bal:.3f} | MCC={mcc:.3f} | n_test={len(test_idx)} | \n
        pos_test={int(y_test.sum())}"
    )

    print(f"\n--- {name} summary ---")
    for k, vals in metrics.items():
        vals = np.asarray(vals, dtype=float)
        print(f"{k}: mean={vals.mean():.3f} std={vals.std():.3f}")

    return metrics

m1 = eval_mode("A) FP-only", use_physchem=False)
m2 = eval_mode("B) FP + physchem", use_physchem=True)

# =====
# 5) FIT FINAL MODEL ON ALL DATA + SAVE (PICKLABLE)
# We save a small dict with:
# - scaler+clf pipeline (sklearn picklable)
# - config needed to rebuild features at inference time
# =====
final_use_physchem = bool(FINAL_USE_PHYSCHEM)

X_all = build_X(X_smiles, use_physchem=final_use_physchem)
final_pipe = make_pipe()
final_pipe.fit(X_all, y, clf__sample_weight=w)

model_bundle = {
    "pipe": final_pipe,
    "use_physchem": final_use_physchem,
    "fp_radius": FP_RADIUS,
    "fp_nbits": FP_NBITS,
    "fp_use_chirality": FP_USE_CHIRALITY,
    "fp_use_features": FP_USE_FEATURES,
    "physchem_dim": 6,
    "threshold": 0.5,
}

joblib.dump(model_bundle, SAVE_MODEL_PATH)
print(f"\n[SAVED MODEL]", SAVE_MODEL_PATH)

# =====
# 6) PROPER ERROR ANALYSIS (NO LEAKAGE!)
# Re-fit fresh model within each fold, generate out-of-fold predictions.
# =====

```

```

rows = []
for fold, (train_idx, test_idx) in enumerate(
    cv.split(X_smiles, y, groups=groups),
    start=1):

    X_train = build_X(X_smiles[train_idx], use_physchem=final_use_physchem)
    X_test = build_X(X_smiles[test_idx], use_physchem=final_use_physchem)

    y_train, y_test = y[train_idx], y[test_idx]
    w_train = w[train_idx]

    fold_pipe = make_pipe()
    fold_pipe.fit(X_train, y_train, clf__sample_weight=w_train)

    proba = fold_pipe.predict_proba(X_test)[: , 1]
    pred = (proba >= 0.5).astype(int)

    for i, idx in enumerate(test_idx):
        true_i = int(y_test[i])
        pred_i = int(pred[i])

        err = "OK"
        if pred_i == 1 and true_i == 0:
            err = "FP"
        elif pred_i == 0 and true_i == 1:
            err = "FN"

        rows.append({
            "fold": fold,
            "molecule_chembl_id": df.iloc[idx]["molecule_chembl_id"],
            "consensus_label": df.iloc[idx]["consensus_label"],
            "smiles": df.iloc[idx]["smiles"],
            "scaffold": df.iloc[idx]["scaffold"],
            "true_label": true_i,
            "pred_label": pred_i,
            "p_active": float(proba[i]),
            "error_type": err,
        })

err_df = pd.DataFrame(rows)
err_df.to_csv(MISCLASS_PATH, index=False)
print("\n[SAVED]", MISCLASS_PATH)
print(err_df["error_type"].value_counts())

# =====
# 7) PER-SCAFFOLD PERFORMANCE (from OOF predictions)

```

```

# =====
scaf_stats = []
for scf, g in err_df.groupby("scaffold"):
    y_true = g["true_label"].values
    y_pred = g["pred_label"].values

    if len(np.unique(y_true)) < 2:
        continue

    scf_stats.append({
        "scaffold": scf,
        "n": int(len(g)),
        "mcc": float(matthews_corrcoef(y_true, y_pred)),
        "bal_acc": float(balanced_accuracy_score(y_true, y_pred)),
        "n_errors": int((g["error_type"] != "OK").sum()),
        "pos_frac": float(np.mean(y_true)),
    })

scaf_df = pd.DataFrame(scaf_stats).sort_values(["mcc", "n"], ascending=[True,
↪False])
scaf_df.to_csv(SCAF_SUM_PATH, index=False)
print("[SAVED]", SCAF_SUM_PATH)
print(scaf_df.head(10))

# =====
# 8) "HARD SCAFFOLDS" REPORT
# =====
hard_scaffolds = scf_df.query("n >= 10 and mcc < 0.5").copy()
hard_scaffolds.to_csv(HARD_SCAF_PATH, index=False)
print("\n[SAVED]", HARD_SCAF_PATH)
print("Hard scaffolds (n>=10 & MCC<0.5):", len(hard_scaffolds))
print(hard_scaffolds.head(10))

# =====
# 9) OPTIONAL: HELPER FOR INFERENCE (LOAD + PREDICT)
# (kept here for convenience; doesn't run unless you call it)
# =====
def load_model_and_predict(smiles_list, model_path=SAVE_MODEL_PATH):
    bundle = joblib.load(model_path)
    pipe = bundle["pipe"]
    use_physchem = bundle["use_physchem"]

    X = build_X(np.asarray(smiles_list, dtype=object),
↪use_physchem=use_physchem)
    proba = pipe.predict_proba(X)[: , 1]

```

```

pred = (proba >= bundle.get("threshold", 0.5)).astype(int)
return proba, pred

```

Loaded & filtered: (2268, 15)

consensus_label

active_single 1502

inactive_single 463

active 286

inactive 17

Name: count, dtype: int64

=== A) FP-only ===

[Fold 1] ROC-AUC=0.979 | PR-AUC=0.994 | BalAcc=0.905 | MCC=0.825 | n_test=227 | pos_test=184

[Fold 2] ROC-AUC=0.989 | PR-AUC=0.998 | BalAcc=0.966 | MCC=0.923 | n_test=227 | pos_test=188

[Fold 3] ROC-AUC=0.980 | PR-AUC=0.997 | BalAcc=0.873 | MCC=0.724 | n_test=227 | pos_test=199

[Fold 4] ROC-AUC=0.916 | PR-AUC=0.984 | BalAcc=0.779 | MCC=0.610 | n_test=227 | pos_test=195

[Fold 5] ROC-AUC=0.965 | PR-AUC=0.974 | BalAcc=0.930 | MCC=0.850 | n_test=227 | pos_test=192

[Fold 6] ROC-AUC=0.942 | PR-AUC=0.969 | BalAcc=0.871 | MCC=0.772 | n_test=227 | pos_test=149

[Fold 7] ROC-AUC=0.957 | PR-AUC=0.972 | BalAcc=0.828 | MCC=0.717 | n_test=227 | pos_test=140

[Fold 8] ROC-AUC=0.961 | PR-AUC=0.990 | BalAcc=0.898 | MCC=0.833 | n_test=227 | pos_test=190

[Fold 9] ROC-AUC=0.936 | PR-AUC=0.988 | BalAcc=0.850 | MCC=0.624 | n_test=226 | pos_test=191

[Fold 10] ROC-AUC=0.923 | PR-AUC=0.961 | BalAcc=0.830 | MCC=0.752 | n_test=226 | pos_test=160

--- A) FP-only summary ---

roc_auc: mean=0.955 std=0.024

pr_auc: mean=0.983 std=0.012

bal_acc: mean=0.873 std=0.052

mcc: mean=0.763 std=0.094

=== B) FP + physchem ===

[Fold 1] ROC-AUC=0.980 | PR-AUC=0.995 | BalAcc=0.896 | MCC=0.823 | n_test=227 | pos_test=184

[Fold 2] ROC-AUC=0.988 | PR-AUC=0.998 | BalAcc=0.966 | MCC=0.923 | n_test=227 | pos_test=188

[Fold 3] ROC-AUC=0.981 | PR-AUC=0.998 | BalAcc=0.891 | MCC=0.748 | n_test=227 | pos_test=199

[Fold 4] ROC-AUC=0.920 | PR-AUC=0.985 | BalAcc=0.779 | MCC=0.610 | n_test=227 | pos_test=195

[Fold 5] ROC-AUC=0.966 | PR-AUC=0.974 | BalAcc=0.944 | MCC=0.868 | n_test=227 | pos_test=192
 [Fold 6] ROC-AUC=0.946 | PR-AUC=0.971 | BalAcc=0.874 | MCC=0.772 | n_test=227 | pos_test=149
 [Fold 7] ROC-AUC=0.957 | PR-AUC=0.972 | BalAcc=0.825 | MCC=0.705 | n_test=227 | pos_test=140
 [Fold 8] ROC-AUC=0.961 | PR-AUC=0.990 | BalAcc=0.898 | MCC=0.833 | n_test=227 | pos_test=190
 [Fold 9] ROC-AUC=0.939 | PR-AUC=0.989 | BalAcc=0.870 | MCC=0.665 | n_test=226 | pos_test=191
 [Fold 10] ROC-AUC=0.928 | PR-AUC=0.964 | BalAcc=0.830 | MCC=0.752 | n_test=226 | pos_test=160

--- B) FP + physchem summary ---

roc_auc: mean=0.957 std=0.022
 pr_auc: mean=0.983 std=0.012
 bal_acc: mean=0.877 std=0.053
 mcc: mean=0.770 std=0.090

[SAVED MODEL] C:\Users\Besitzer\Desktop\M3_Project\M3_databases\m3_fp_physchem_scaffolddcv.joblib

[SAVED] C:\Users\Besitzer\Desktop\M3_Project\M3_databases\scaffold_cv_misclassifications.csv

error_type

OK 2099

FP 110

FN 59

Name: count, dtype: int64

[SAVED] C:\Users\Besitzer\Desktop\M3_Project\M3_databases\scaffold_performance_summary.csv

	scaffold	n	mcc	bal_acc	\
3	<chem>O=C(NCC1CCNCC1)c1ccccc1-c1ccccc1</chem>	4	-1.000000	0.000000	
4	<chem>O=C(NCc1cccc(-c2cccc(CN3CCNCC3)c2)c1)c1ccc2c(c...</chem>	10	-0.333333	0.222222	
13	<chem>c1ccc2c(c1)CCc1cccnc1C2=C1CCNCC1</chem>	4	-0.333333	0.333333	
11	<chem>c1ccc(C(CC2CC3CCC(C2)[NH2+])3)c2ccccc2)cc1</chem>	29	0.000000	0.500000	
5	<chem>O=C(Nc1ccccc1)NC(Cc1ccccc1)C(=O)NC1CCN(Cc2ccccc...</chem>	17	0.000000	0.500000	
16	<chem>c1ccccc1</chem>	12	0.000000	0.500000	
15	<chem>c1ccc2c(c1)Nc1ccccc1S2</chem>	5	0.000000	0.500000	
7	<chem>O=C(OC1CN2CCC1CC2)N1CCc2ccccc2C1c1ccccc1</chem>	4	0.000000	0.500000	
14	<chem>c1ccc2c(c1)CNC(CNC1CCCc3cccnC31)C2</chem>	4	0.000000	0.500000	
9	<chem>O=C1CCc2ccccc2N1CCCN1CCCC1</chem>	3	0.000000	0.500000	

	n_errors	pos_frac
3	4	0.500000
4	6	0.900000
13	2	0.250000
11	2	0.931034

5	1	0.941176
16	9	0.250000
15	1	0.800000
7	1	0.250000
14	1	0.250000
9	2	0.333333

[SAVED] C:\Users\Besitzer\Desktop\M3_Project\M3_databases\hard_scaffolds.csv
Hard scaffolds (n>=10 & MCC<0.5): 5

	scaffold	n	mcc	bal_acc	\
4	O=C(NCc1cccc(-c2cccc(CN3CCNCC3)c2)c1)c1ccc2c(c...	10	-0.333333	0.222222	
11	c1ccc(C(CC2CC3CCC(C2)[NH2+])3)c2cccc2)cc1	29	0.000000	0.500000	
5	O=C(Nc1cccc1)NC(Cc1cccc1)C(=O)NC1CCN(Cc2cccc...	17	0.000000	0.500000	
16	c1cccc1	12	0.000000	0.500000	
12	c1ccc2c(c1)CCN1Cc3c(ccc4[nH]ccc34)OC21	18	0.152286	0.564935	

	n_errors	pos_frac
4	6	0.900000
11	2	0.931034
5	1	0.941176
16	9	0.250000
12	9	0.611111

[]: